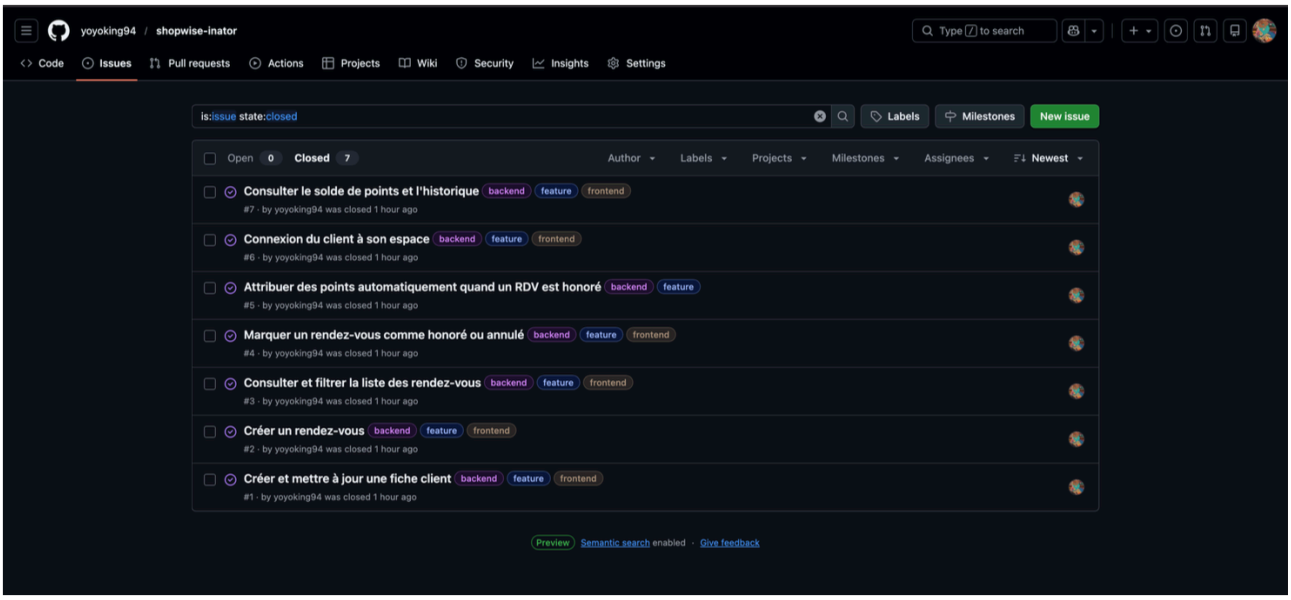
Shopwise

1. Suivi des taches
2. Base de données
3. Code source
4. Code source des tests
5. Pipeline CI/CD
6. Dockerfile
7. Docker-compose.yml
8. Readme.md

Suivi des taches



Base de données

```sql
-- drop database shopwise;
CREATE DATABASE IF NOT EXISTS shopwise;
USE shopwise;
-- Table unique utilisateur (remplace client + utilisateur)
CREATE TABLE utilisateur (
id INT AUTO_INCREMENT PRIMARY KEY,
nom VARCHAR(100) NOT NULL,
email VARCHAR(150) NOT NULL UNIQUE,
mot_de_passe VARCHAR(255) NOT NULL,
telephone VARCHAR(20),
date_creation DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
role ENUM('CLIENT'
,
'COMMERCANT') NOT NULL
);
-- Table rendez_vous (client_id pointe maintenant vers
utilisateur)
CREATE TABLE rendez_vous (
id INT AUTO_INCREMENT PRIMARY KEY,
utilisateur_id INT NOT NULL,
date_rendez_vous DATE NOT NULL,
heure_rendez_vous TIME NOT NULL,
service VARCHAR(100) NOT NULL,
statut ENUM('EN_ATTENTE'
,
'HONORE'
,
'ANNULE') NOT NULL DEFAULT
'EN_ATTENTE'
,
FOREIGN KEY (utilisateur_id) REFERENCES utilisateur(id)
);
-- Table points_fidelite
CREATE TABLE points_fidelite (
id INT AUTO_INCREMENT PRIMARY KEY,
utilisateur_id INT NOT NULL UNIQUE,
solde_points INT NOT NULL DEFAULT 0,
FOREIGN KEY (utilisateur_id) REFERENCES utilisateur(id)
);
-- Table transaction_fidelite
CREATE TABLE transaction_fidelite (
id INT AUTO_INCREMENT PRIMARY KEY,
utilisateur_id INT NOT NULL,
rendez_vous_id INT NOT NULL,
points_attribues INT NOT NULL,
date_transaction DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (utilisateur_id) REFERENCES utilisateur(id),
FOREIGN KEY (rendez_vous_id) REFERENCES rendez_vous(id)
);
```

```sql
-- Données de testINSERT INTO utilisateur (nom, email,
mot_de_passe, telephone,
role) VALUES
('Marie Dupont'
,
'commercant@shopwise.com'
,
'shopwise123'
,
'0600000000'
,
'COMMERCANT'),
('Alice Martin'
,
'alice@email.com'
,
'alice123'
,
'0601020304'
,
'CLIENT'),
('Bob Durand'
,
'bob@email.com'
,
'bob123'
,
'0605060708'
,
'CLIENT'),
('Clara Petit'
,
'clara@email.com'
,
'clara123'
,
'0609101112'
,
'CLIENT');
INSERT INTO rendez_vous (utilisateur_id, date_rendez_vous,
heure_rendez_vous, service, statut) VALUES
(2,
'2026-03-05'
,
'10:00:00'
,
'Coupe cheveux'
,
'HONORE'),
(3,
'2026-03-06'
,
```

```
'14:30:00'
,
'Coloration'
,
'EN_ATTENTE'),
(4,
'2026-03-07'
,
'09:00:00'
,
'Soin visage'
,
'ANNULE');
INSERT INTO points_fidelite (utilisateur_id, solde_points) VALUES
(2, 10),
(3, 0),
(4, 0);
INSERT INTO transaction_fidelite (utilisateur_id, rendez_vous_id,
points_attribues) VALUES
(2, 1, 10);
select * from utilisateur;
```

Code source

/Shopwise/frontend
/Shopwise/backend

Code source test

Karma v 6.4.4 - connected; test: complete;

Chrome 145.0.0.0 (Mac OS 10.15.7) is idle

Jasmine 4.6.1

30 specs, 0 failures, randomized with seed 65327

```
EspaceClient
  • should create
Fidelisation
  • should be created
Utilisateur
  • should be created
Connexion
  • ne soumet pas si le formulaire est invalide
  • soumettre avec identifiants valides redirige le commerçant
  • soumettre avec identifiants valides redirige le client
  • should create
  • soumettre avec mauvais identifiants affiche un message derreur
App
  • should create the app
  • should render title
RendezVous
  • should be created
Connexion
  • should be created
ListeUtilisateur
  • allerVersNouveauClient navigue vers /clients/nouveau
  • should create
FormulaireUtilisateur
  • annuler navigue vers /clients
  • soumettre ne fait rien si le formulaire est invalide
  • le formulaire est invalide si les champs sont vides
  • should create en mode création
  • soumettre en mode création appelle creerClient
  • le formulaire est valide avec les champs requis remplis
DetailFidelisation
  • retourVersClients navigue vers /clients
  • should create
FormulaireRendezVous
  • should create
ListeRendezVous
  • chargerRendezVous filtre par statut si filtreStatut est défini
  • chargerRendezVous filtre par date si filtreDate est défini
  • should create
  • chargerRendezVous filtre par client si filtreClientId est défini
  • reinitialiserFiltres remet tous les filtres à vide
  • allerVersNouveauRendezVous navigue vers /rendez-vous/nouveau
  • changerStatut appelle le service avec le bon id et statut
```

```
Chrome 145.0.0.0 (Mac OS 10.15.7): Executed 30 of 30 SUCCESS (0.453 secs / 0.401 secs)
TOTAL: 30 SUCCESS

=============================== Coverage summary ===============================
Statements   : 72.72% ( 128/176 )
Branches     : 63.63% ( 14/22 )
Functions    : 61.53% ( 48/78 )
Lines        : 73.25% ( 126/172 )
```

backend

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.shopwise.backend.mapper		0 %		n/a	12	12	12	12	12	12	3	3
com.shopwise.backend		37 %		n/a	1	2	2	3	1	2	0	1
com.shopwise.backend.service		100 %		83 %	1	23	0	86	0	20	0	4
com.shopwise.backend.controller		100 %		100 %	0	13	0	16	0	10	0	4
com.shopwise.backend.config		100 %		n/a	0	2	0	5	0	2	0	1
com.shopwise.backend.entities		100 %		n/a	0	2	0	7	0	2	0	2
Total	65 of 611	89 %	1 of 12	91 %	14	54	14	129	13	48	3	15

Pipeline CI/CD

```
name: Intégration Continue ShopWise
on:
push:
branches: [ main, develop ]
pull_request:
branches: [ main, develop ]
jobs:
tests-backend:
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v4
- uses: actions/setup-java@v4
with:
java-version: '17'
distribution: 'temurin'
- name: Cache Maven
uses: actions/cache@v4
with:
path: ~/.m2/repository
key: ${{ runner.os }}-maven-${{ hashFiles('**/
pom.xml') }}
- run: mvn clean verify jacoco:report
working-directory: ./backend
- uses: actions/upload-artifact@v4
with:
name: jacoco-backend-report
path: backend/target/site/jacoco/
tests-frontend:
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v4
- uses: actions/setup-node@v4
with:
node-version: '20'
cache: 'npm'
cache-dependency-path: frontend/package-lock.json
- name: Install Frontend
run: npm ci
working-directory: ./frontend- name: Run Angular Tests
run: npm run test:ci
working-directory: ./frontend
- uses: actions/upload-artifact@v4
if: always()
with:
name: angular-coverage-${{ github.run_id }}
path: frontend/coverage/
docker-build:
runs-on: ubuntu-latest
needs: [tests-backend, tests-frontend]
```

```
steps:
- uses: actions/checkout@v4
- name: Login DockerHub
uses: docker/login-action@v3
with:
username: ${{ secrets.DOCKER_USERNAME }}
password: ${{ secrets.DOCKER_PASSWORD }}
- run: docker build -t yoyoking94/shopwise-backend:latest ./
backend
- run: docker build -t yoyoking94/shopwise-
frontend:latest ./frontend
- run: docker push yoyoking94/shopwise-backend:latest
- run: docker push yoyoking94/shopwise-frontend:latest
deploy-quality:
runs-on: ubuntu-latest
needs: docker-build
if: github.ref == 'refs/heads/develop'
steps:
- run: echo "Déploiement staging OK ! »
```

Dockerfile

Frontend :

```
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=builder /app/dist/frontend/browser /usr/share/nginx/
html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Backend :

```
FROM maven:3.9-eclipse-temurin-21 AS etape-compilation
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=etape-compilation /app/target/*.jar application.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "application.jar"]
```

Docker-compose

```yaml
services:
  mysql:
    image: mysql:8.0
    container_name: shopwise-mysql
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: shopwise
    ports:
      - "3307:3306"
    volumes:
      - mysql_data:/var/lib/mysql
      - ./backend/src/main/resources/data.sql:/docker-entrypoint-
initdb.d/1-data.sql
    healthcheck:
      test: ["CMD-SHELL", "mysqladmin ping -h localhost -u root
-proot || exit 1"]
      interval: 20s
      timeout: 10s
      retries: 20
      start_period: 40s

  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    container_name: shopwise-backend
    restart: always
    ports:
      - "8080:8080"
    environment:
      SPRING_DATASOURCE_URL: jdbc:mysql://mysql:3306/shopwise?
createDatabaseIfNotExist=true&useSSL=false&serverTimezone=UTC&allo
wPublicKeyRetrieval=true
      SPRING_DATASOURCE_USERNAME: root
      SPRING_DATASOURCE_PASSWORD: root
      SPRING_JPA_HIBERNATE_DDL_AUTO: update
    depends_on:
      mysql:
        condition: service_healthy

  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    container_name: shopwise-frontend
    restart: always
    ports:
      - "4200:80"
```

```
    depends_on:
      - backend

volumes:
  mysql_data:
```

Readme.md

```
<img width="1758" height="1050" alt="Capture d’écran 2026-03-03 à
16 55 41" src="https://github.com/user-attachments/assets/
e1f68e1e-f553-43c7-aee2-ece91436f5ec" />


Shopwise - Procédure de déploiement Docker & CI/CD

    -   Présentaion :
        Shopwise est une application web composée d’un backend
Spring Boot, d’un frontend Angular, et d’une base de données
MySQL.
        L'ensemble est dockerisé pour simplifier le développement,
la montée en production et le déploiement continu.

    -   Prérequis :
        Docker installé sur votre ordinateur (Windows/Mac/Linux)
        Accès à Docker Hub (compte gratuit)
        Accès au dépôt GitHub : shopwise

    -   Les captures d'écran sont dans le dossier livrable

    1.  Contruction des images Docker
        Les Dockerfiles sont inclus dans chaque dépôt :
        Backend : backend/Dockerfile
        Frontend : frontend/Dockerfile

        Directement à la racine du projet (dossier shopwise)
        Pour construire et lancer les services :
            docker-compose down -v && docker-compose build &&
docker-compose up -d

        Lancer l'ensembre avec :
            docker-compose up

        Arrêtez l'ensemble avec :
            docker-compose down

    2. Accéder à l’application
        Frontend : http://localhost:4200
        Backend : http://localhost:8080
        MySQL : localhost:3307

        PS : si vous souhaitez créer un client, pour recuperer son
mot de passe, vous devrez aller le recuperer dans la base de
données.

    3. Déploiement continu - GitHub Actions
```

Chaque dépôt dispose d'un workflow CI/CD pour builder et pousser les images Docker sur Docker Hub automatiquement à chaque commit sur la branche principale (main).